

SATA-DRL: 의존 작업의 실행 위치를 학습하다

Task Graph offloading via Deep Reinforcement Learning in Mobile Edge Computing

Jiagang Liu · Yun Mi · Xinyu Zhang · Xiaocui Li (2024)

arrival·completion event로 ready task를 꺼내고, 시간 변화 ECD 상태에서 한 ECD를 선택한다

문제: 독립 task 하나가 아니라 의존 DAG를 배치한다

- 모바일 application은 여러 atomic task와 그 사이의 data dependency로 구성된다.
- 부모 출력이 도착하고 ECD가 비어야 자식 task의 실행을 시작할 수 있다.
- ECD의 processing capability와 queue workload가 시간에 따라 달라진다.

현재 task의 ECD 선택은 나중 task의 data arrival과 queue를 바꾼다.

논문을 읽기 위한 용어

약어	풀어 쓰기	이 발표에서의 역할
MU / ECD	Mobile User / Edge Computing Device	단말 / 계산 목적지
PE / DAG	Processing Element / Directed Acyclic Graph	계산 장치 / 의존 그래프
MDP / DQN	Markov Decision Process / Deep Q-Network	의사결정 모델 / Q 학습
SATA-DRL	Scheduling strAtegy for the Task grAph	DQN 기반 작업 배치

MEC system: 중앙 controller와 ECD queue

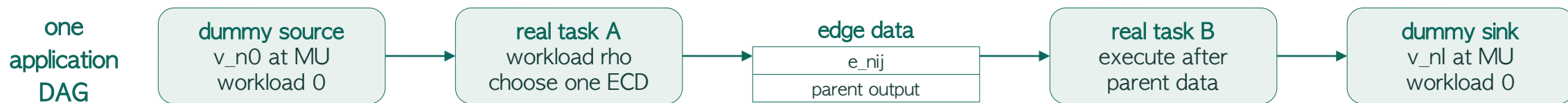
controller는 ECD status를 받고, task scheduling decision을 한 곳에서 내린다



이 논문의 agent는 decentralized vehicle agent가 아니라 중앙 controller에 있다.

DAG의 source·real task·sink

논문은 MU를 fictitious ECD 0으로 두고 dummy source와 sink를 추가한다



real task는 local 실행 선택지가 없고, 정확히 한 real ECD에 배치된다.

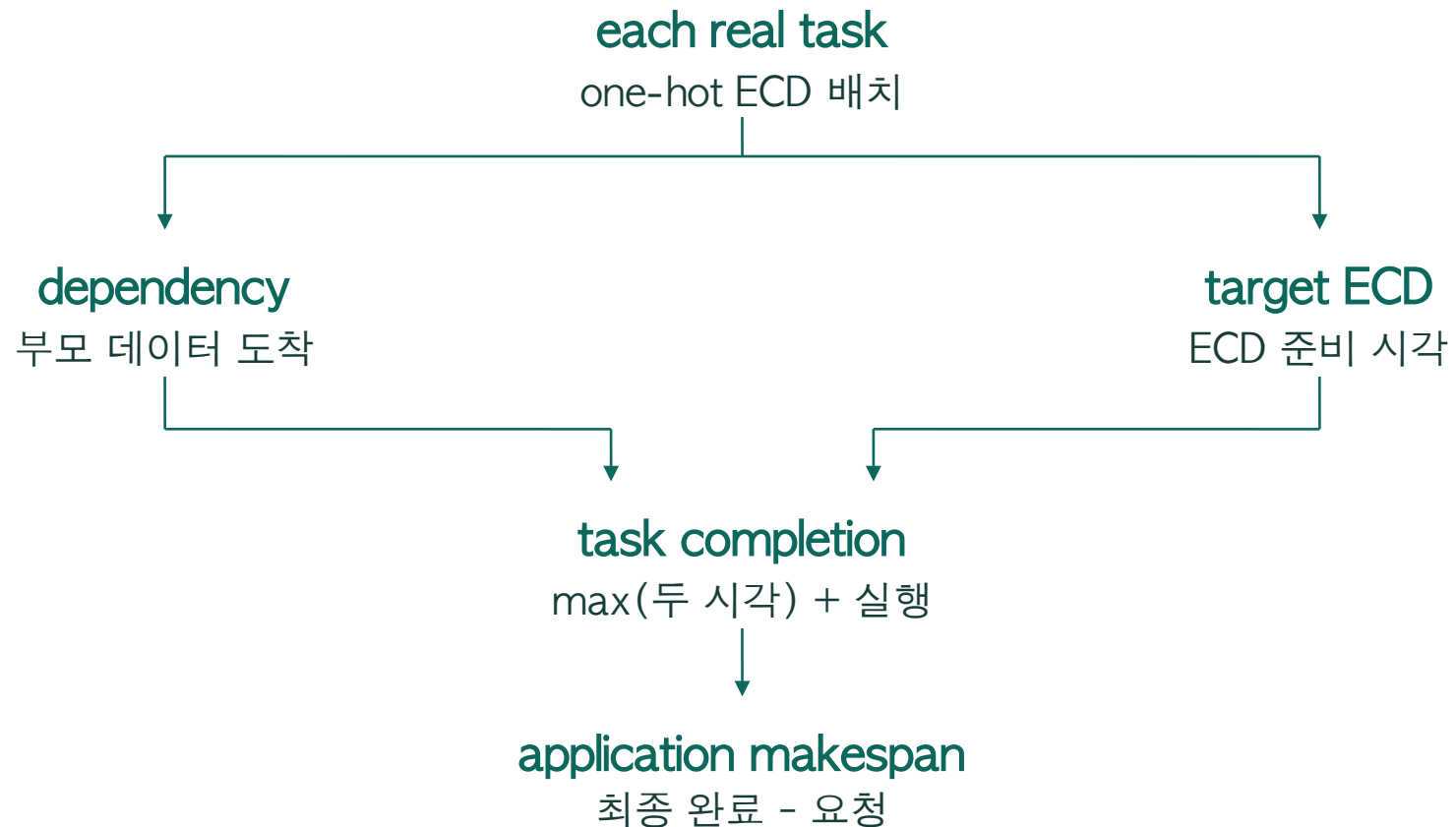
완료 시간: 입력 도착·큐 준비·실행

요소	언제 생기는가	논문 모델
데이터 전송	부모와 다른 ECD에 배치	부모 완료 뒤 링크 전송
큐 대기	목적지 ECD가 바쁨	ECD 준비 시각까지 대기
실행	입력·ECD 모두 준비	작업량 / ECD 처리 능력
같은 ECD	부모·자식이 같은 ECD	두 작업 사이 전송 지연 0

child completion은 가장 늦은 parent data arrival과 target queue를 함께 기다린다.

목표: application 평균 makespan 최소화

makespan은 요청 시각 뒤 모든 task가 끝날 때까지의 길이다



SATA는 arrival-completion event에서 움직인다

01	이벤트	입력 도착 / 작업 완료
↓		
02	도착 시 목록	F_lct 오름차순
↓		
03	준비 항목	각 목록 맨 앞 검사
↓		
04	다음 작업	Q_r에서 하나 선택
↓		
05	ECD 배치	DQN으로 목적지 결정

DQN은 임의 DAG 전체를 한 번에 배치하지 않고, ready task 하나씩 결정한다.

F_lct: 기한에서 거꾸로 계산한 우선순위

가상 $A \rightarrow B$, $A \rightarrow C$; $\text{deadline}=20$. (B,C) 의 반환= $(2,1)$, 실행= $(3,4)$, A로부터 전송= $(1,2)$

항목	Eq. 17의 계산	의미
F_lct(B)	$20-2 = 18$	최종 반환 전에 B 종료
F_lct(C)	$20-1 = 19$	최종 반환 전에 C 종료
F_lct(A)	$\min(18-3-1, 19-4-2) = 13$	두 자식 중 더 빠른 요구
ready 순서	B와 C가 모두 ready이면 $B \rightarrow C$	작은 F_lct부터 선택

F_lct는 보장된 실제 완료 시각이 아니라 빠른 자원 기준으로 추정된 기한이다.

배치를 결정하는 시점과 실행 시작은 다르다

구분	원문에 있는 조건	설명 범위
p. 6 ready	부모의 scheduling plan 이후	배치 순서의 조건
p. 7 보상 설명	ready의 부모가 완료됨을 가정	본문 표현 차이 존재
Algorithm 1	각 목록의 맨 앞 ready만 수집	모든 DAG 정점 동시 탐색 아님
실제 실행 시작	$\max(\text{ECD 준비}, \text{모든 입력 도착})$	Eq. 12가 정하는 시각

ready를 곧바로 데이터 도착 완료 또는 실행 시작이라고 부르지 않는다.

MDP state: 다섯 종류의 지연 관련 관측

state component	무엇을 요약하는가	결정에 필요한 이유
inter-ECD rate	ECD 간 전송률의 합	parent output 이동 시간
MU link rate	MU와 covering ECD link	application input·result 경로
ECD capability	ECD 처리 능력의 합	execution time
ready workload	Q _r 의 총 workload	곧 배치할 작업량
계산 큐 작업량	전체 ECD 큐 작업량 합	FCFS 대기 시간

state는 DAG 모양 전체가 아니라 link·capacity·queue의 집계 관측이다.

합계가 같아도 어느 ECD가 빠른지는 다를 수 있다

교육용 두 ECD, 빈 큐·전송 영향 없음, 작업량 6000 MI; 다른 관측은 동일하게 둔다

상황	ECD 1 / ECD 2 MIPS	관측된 합계	실행 시간
A	6000 / 4000	10000 MIPS	ECD 1: 1 s, ECD 2: 1.5 s
B	4000 / 6000	10000 MIPS	ECD 1: 1.5 s, ECD 2: 1 s
차이	빠른 ECD가 바뀜	합계 관측은 같음	유리한 선택은 바뀜

집계 state만으로 모든 개별 자원 상태를 구분한다고 보장할 수 없다.

action: 현재 ready task를 한 ECD에 배치

one-hot binary action은 task의 target ECD 하나를 뜻한다



action은 CPU allocation이나 bandwidth allocation이 아니라 task-to-ECD assignment다.

$$\text{reward} = \text{utility} - \text{duration} - \text{penalty}$$

항	원문 정의	직관
U	$\beta \log_2(\rho)$	작업량 utility
D	$\psi (\max A + Q + E)/\rho$	합으로 만든 지연 비용
P	$\eta (F - F_{lct})/\rho$	일찍 끝나면 음수 가능
r	$U - D - P$	원문 보상 정의

reward는 makespan 하나가 아니라 workload 보정 utility와 두 종류의 지연 비용을 결합한다.

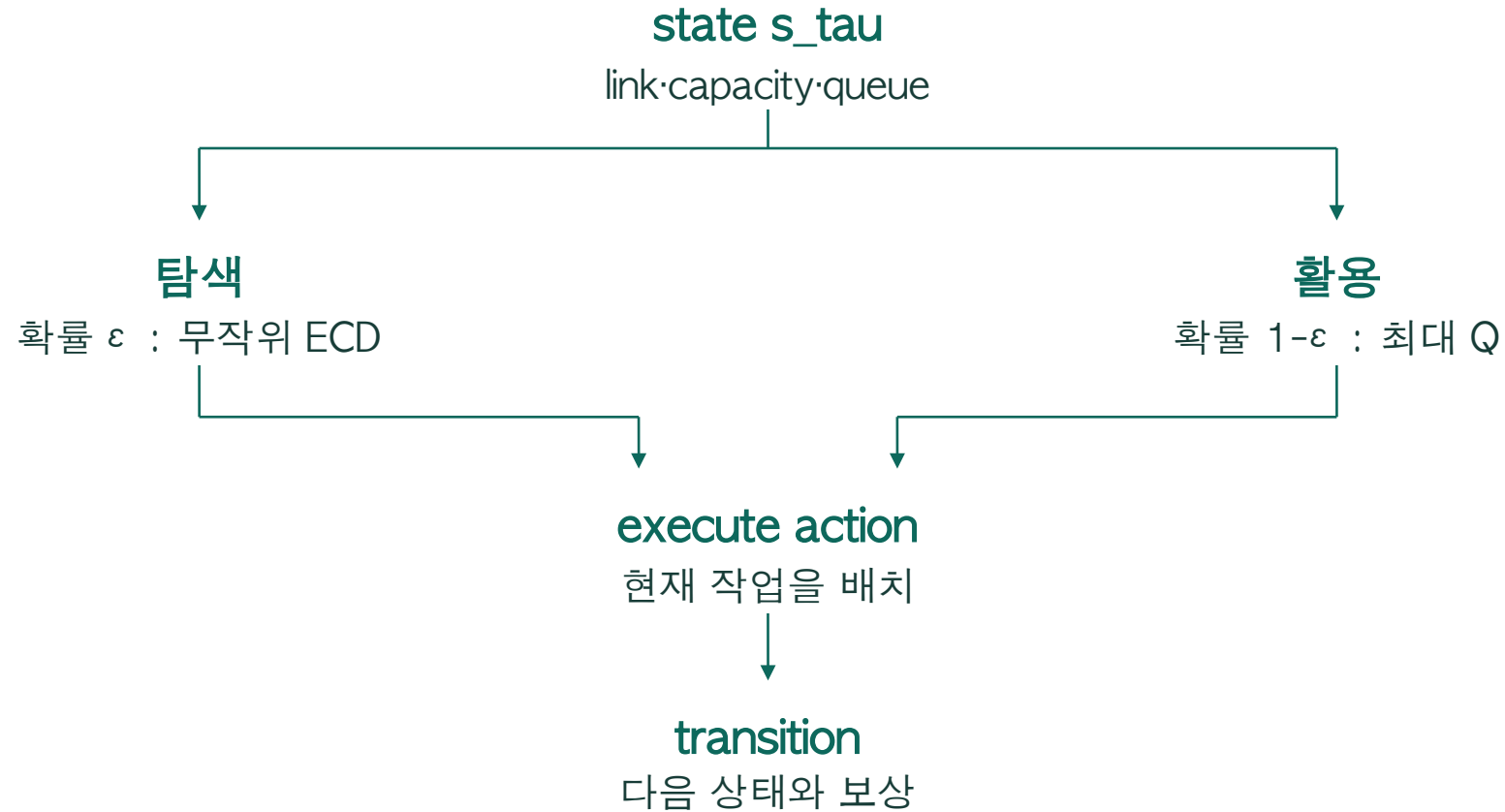
교육용 계산: 완료 시간과 shaped reward를 구분

가상 값 $\rho = 100$, $\max A = 2$, $Q = 3$, $E = 1$, $F_{\text{ict}} = 5$; 원문 가중치 $\beta = .6$, $\psi = 5$, $\eta = 40$

계산	대입	결과
실제 완료 F	$\max(3, 2) + 1$	4
utility U	$.6 \log_2(100)$	약 3.9863
duration D	$5(2+3+1)/100$	0.30
penalty P / reward r	$40(4-5)/100$; $U - .30 - P$	-0.40 / 약 4.0863

기한 추정보다 빠르면 $P < 0$ 이다; 보상식의 D는 F/ρ 와 다르다.

training action은 epsilon-greedy로 고른다



DQN training: replay, target, loss

$Q_{tar} = r + \gamma \max_a Q_{target}(s', a)$; 학습하는 값은 $Q_{current}(s, a)$ 다



prediction network는 current Q, target network는 bootstrap target을 제공한다.

target network: K회 반복 뒤 파라미터 복사

- prediction network θ 는 minibatch 손실을 줄이는 방향으로 갱신한다.

- Algorithm 2의 $\theta \leftarrow \theta$ 는 $k=1 \cdots K$ 반복문 바깥에 있다.

- Fig. 4와 본문도 동기화 조건을 만족할 때 target에 복사한다고 설명한다.

K회 처리 블록과 복사를 구분한다; 매 scheduling 단계 복사라고 읽지 않는다.

Algorithm 1과 2의 실제 연결



SATA가 scheduler의 순서를 만들고, DQN이 각 ready task의 ECD 선택을 학습한다.

학습 뒤 SATA-Work의 결정

01	학습 반복	여러 episode 경험
↓		
02	학습된 θ	prediction 사용
↓		
03	이벤트	ready 항목 수집
↓		
04	최대 Q	탐색 없는 선택
↓		
05	작업 배치	ECD queue에 할당

working 단계는 exploration 대신 prediction network의 argmax action을 쓴다.

교육용 가상 trace: parent output이 child 시작을 바꾼다

가상 값: workload는 instructions, 속도는 instructions/s; 대기 경쟁·초기 입력 전송은 제외한다

가정	A: ECD 1	B: ECD 2	B: ECD 1
작업량 / 속도	1000 / 1000	1000 / 1000	1000 / 1000
A 완료	1 s	입력 200 bit	입력 200 bit
부모 전송	가정: 초기 입력 도착	$200 / 200$ $= 1\text{ s}$	같은 ECD: 0 s
B 완료	해당 없음	$\max(0, 2) + 1$ $= 3\text{ s}$	$\max(1, 1) + 1$ $= 2\text{ s}$

이 가정에서 same-ECD placement는 inter-ECD parent transfer를 피한다.

실험 조건: 네 ECD와 Montage 작업 그래프

항목	논문 설정	항목	논문 설정
real ECD	4	PE	ECD마다 1
ECD capability	4000–6000 MIPS	inter-ECD rate	440 Mbps
MU covering link	1000 Mbps	workflow	Montage 25 nodes
deadline	$r_n + 6 MS_n$	capacity dynamics	stated Markov matrix
simulator	CloudSim 확장	evaluation	simulation

결과는 네 ECD, 정량화된 capacity, Montage workflow라는 simulation 조건에 묶인다.

deadline metric을 읽을 때의 조건

- 논문은 reference makespan MS_n 을 계산한 뒤 deadline을 $r_n + 6 MS_n$ 으로 둔다.
- MS_n 은 각 task를 서로 다른 가상 ECD에 5000 MIPS로 배치하고 전송을 무시한 기준이다.
- deadline violation은 deadline을 넘긴 application 수 / 전체 application 수다.

deadline violation은 외부 SLA가 아니라 논문이 만든 deadline 규칙의 결과다.

DQN agent의 보고된 설정

구성	값	구성	값
완전 연결층	5개 층	뉴런 수	128→64→32→16→5
optimizer	Adam	learning rate	0.0006
experience pool	200000	minibatch	64
β / ψ / η	0.6 / 5 / 40	할인율 γ	0.95

이 설정은 논문 simulation의 재현 단서이며 다른 task graph의 최적값 보장은 아니다.

재현할 때 원문만으로 확정하기 어려운 지점

지점	보고된 내용	남는 질문
출력 / 행동	4 ECD, 출력 5개, $a_0 \cdots a_4$	real task의 a_0 금지 구현은?
출력 범위	앞 4층 linear, 마지막 softmax	Q-target의 값 범위 처리는?
기준 5000	$5000 = (6000 \times 4 + 1000) / 6$	우변은 약 4166.67
학습 호출	K회 뒤 $\theta \leftarrow \theta$; episode 반복	호출 간 buffer·상태 지속은?

모델 제약과 보고된 설정을 구분하고, 미제공 구현은 추정으로 채우지 않는다.

cumulative reward 학습 곡선의 범위

- arrival rate λ 5, 7, 9에서 episode당 10 application을 simulator에 넣는다.
- Fig. 5는 cumulative reward가 episode와 함께 증가하는 세 곡선을 보인다.
- 본문은 Fig. 5(c)에서 600 episode 이후 상대적으로 안정적이라고 해석한다.

이 곡선은 arrival-rate별 학습 관찰일 뿐, 일반 convergence 증명은 아니다.

optimization 비교의 대상과 측정값

방법	논문에서의 역할	두 지표 공통
Zhang's PCP	휴리스틱 비교	makespan·기한 위반
OnDoc	온라인 DAG 비교	makespan·기한 위반
DTO-CED	의존 작업 비교	makespan·기한 위반
Dueling DQN	같은 상태·행동·보상	makespan·기한 위반
SATA-DRL	제안 방법	makespan·기한 위반

모든 방법은 같은 fetched application을 30회 처리한 평균으로 비교된다.

Fig. 6–7의 조건부 결과

그림	실험 조건	본문의 보고
Fig. 6	작업 그래프 5–35개 $\lambda = 5, 7, 9$	SATA-DRL의 makespan 최소
Fig. 6	같은 simulation	PCP는 부하 증가에 민감
Fig. 7	작업 그래프 5–35개 $\lambda = 5, 7, 9$	SATA-DRL의 위반율 더 낮음
Fig. 6–7	Dueling DQN 비교	이 설정에서 SATA-DRL 우세

논문은 plotted settings에서 SATA-DRL의 makespan-deadline violation이 낮다고 보고한다.

모델을 적용할 때 남는 경계

경계	원문 모델에서의 처리	해석상 주의
control	central controller가 status 수집	distributed robustness 미검증
compute	ECD당 one PE, FCFS	parallel scheduler와 다름
capacity	5 quantized MIPS levels	continuous runtime dynamics 아님
목표 / 평가	makespan 최소화 / 기한 위반 측정	energy·cost 최적화 아님
deadline	transfer-ignoring MS_n에서 구성	external SLA로 일반화 불가

SATA-DRL은 명시된 simulation MEC model에서의 task-to-ECD scheduling 방법이다.

SATA-DRL의 전체 연결

01	DAG	부모 출력 의존성
↓		
02	SATA	이벤트·우선순위
↓		
03	MDP	집계 상태·ECD 선택
↓		
04	DQN	경험으로 Q 학습
↓		
05	결과	simulation 조건부

이 논문은 의존 task의 event-driven 순서와 DQN의 ECD 선택을 결합한다.